Team Project Phase 2 Report

Summary

Team name: dwendwen
Members (names and Andrew IDs):

Qing Deng (qingd)
Jiarui Zhang (jiaruiz)
Wangshu Li (wangshul)

Live Test Performance & Configurations

Database chosen (MySQL/HBase):
Number and types of instances: seven m5a.large instances (one master, six worker)
Cost per hour of the entire system:
EC2: 0.086 * 7 = 0.602
EBS: (20*2 + 32*7 + 39*3) * 0.1/720 = 0.0529
ELB: 0.0225 * 2 = 0.045
Total: $0.6999 / hour
(Cost includes on-demand EC2, EBS and ELB costs. Ignore S3, EMR software addition, Network and Disk I/O costs)

Your team's overall rank on the live test scoreboard:
Live test submission details:

	M1	M2	M3
score	7.04	20	2.06
submission id	717722	717723	717724
throughput	38151.14	31244.56	1028.94
latency	10.76	11.56	76.99
correctness	100.00	88.05	12.71
error	0.00	0.00	0.00

Rubric

- Each unanswered bullet point = -4%
- Each unsatisfactory answer = -2%

- Use the report as a record of your progress, and then condense it before sharing with us.
- If you write down an observation, we also expect you to try and explain it.
- Questions ending with "Why?" need evidence (not just logic).
- Always use your own words (paraphrase); we will check for plagiarism.

Task 1: Improvements of Microservices

Question 1: Cluster architecture

You are allowed to use several types of EC2 instances, and you are free to distribute the workload across your cluster in any way you'd like (for example, hosting MySQL only on some worker nodes, or having worker nodes of different sizes). When you try to improve the performance of your microservices, it might be helpful to think of how you make use of the available resources within the provided budget.

- List at least two reasonable varieties of cluster architectures. (e.g., draw schematics to show which pieces are involved in serving a request)
  1. Hosting MySQL pods in one or two specific instances, while hosting web-tier pods in the other instances.
  2. Distributing all pods evenly across all instances.
- Discuss how your design choices mentioned above affect performance, and why.
  Separating database and web workloads reduces CPU and I/O contention. Independent ASP.NET Core services isolate the CPU-heavy m1/m2 handlers from m3 database access; the combined Java application shares one process. Pod placement, connection pools and replica counts determine resource use.
- Describe your final choice of instances and architecture for your microservices. If it is different from what you had in Phase 1, describe whether the performance improved and why.
  We have 6 worker nodes. We deployed 6 web-tier pods, evenly distributed into these 3 worker nodes. We also deployed 3 MySQL pods, which were distributed to 3 random worker nodes. For Phase 1, we only deployed one MySQL pod. Now we use a stateful set of three MySQL pods, which can improve performance.

Question 2: Database and schema

If you want to make Microservice 3 faster, the first thing to look at is the database because the amount of computation at the web-tier is not as much as in Microservice 1 and 2. As you optimize your Microservice 3, you will find out various schemas can result in a very large difference in performance! Also, you might want to look at different metrics to understand the bottleneck and think of what to optimize.

- What schema did you use for Microservice 3 in Phase 1? Did you change it in Phase 2? If so, please discuss how and why you changed it, and how did the new schema affect performance?
  1. We join the interaction score table and hashtag score table to a single table. This will reduce the query time because we just query once for both hashtag score and interaction score.

2. We delete all unnecessary rows in the user table, specifically, we only keep the latest information so we delete all unnecessary rows to make the table as small as possible. This will save space as well as reduce the query time.
3. We also plan to merge all our tables into one table to further reduce query time in Phase3.

- Try to explain why one schema is faster than another.

  Repeated queries inside a contact loop increase latency. A per-user serving record supports one retrieval followed by local scoring. The C# ContactRepository queries contact_tweets by parameterized user_id and deserializes contact_tweet JSON. TwitterService merges reply/retweet records and multiplies interaction, hashtag and keyword scores. KMP counts overlapping phrase matches; full hashtag matching is case-insensitive. Results sort by score, then numeric user ID, both descending. This aligns retrieval with the response rather than raw tweets.

- Explain briefly the theory behind at least 3 performance optimization techniques (different from the ones in your Phase 1 report) for databases in general. Are they related to the problem you found above?
  1. Optimize MySQL query statement, using EXPLAIN statements to inspect the number of rows to be scanned. If one query scans less number of rows, it executes faster.
  2. Pre-store each user's latest information (screen name, description), instead of searching when requests come. This avoids doing ORDER BY when querying each user's latest screen name and description, which saves time.
  3. Deploy 3 MySQL pods instead of one pod. Pre-storing each user's latest information More replicated databases can receive more requests per second.
- How is each of these optimizations implemented in your storage-tier? Which optimizations only exist in only one of MySQL and HBase and why? How can you simulate that optimization in the other (or if you cannot, why not)?

  We just followed the step described in the previous question to implement them in our storage-tier.

  Adding indexes to MySQL database tables only exists in MySQL because it uses a B+ tree to store the data in the database. So instead of looking up the whole table, indexes can help to find the queried data quickly. We can use the DESCRIBE command in mysql console before and after adding indexes. If the index is correctly set, the number of rows needed to be looked up will significantly decrease. For HBase, we can optimize the row key design to speed up the query.

Question 3: Your ETL process

It's highly likely that you need to re-run your ETL process in Phase 2 each time you implement a new database schema. You might even find it necessary to re-design your ETL pipeline if your previous one is not sufficient for your needs. For the following 9 bullet points, please briefly explain:

- The programming model/framework used for the ETL job and justification.

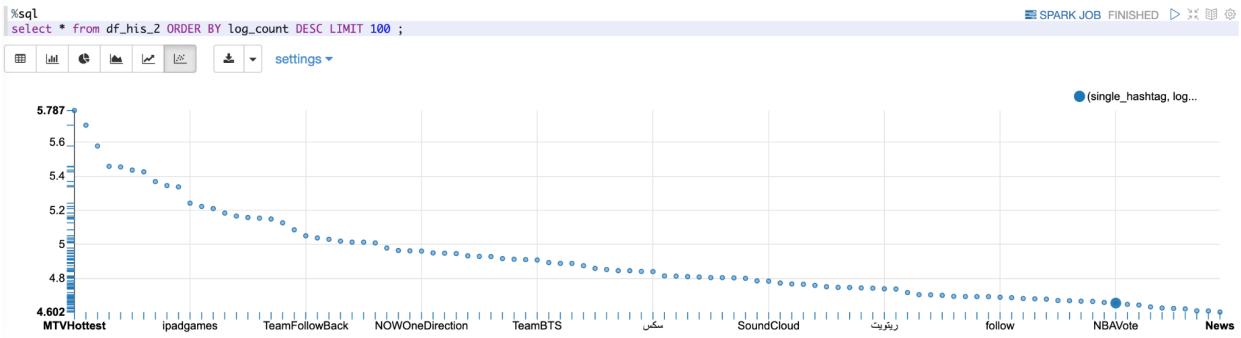
  We used scale/spark for the ETL job. We also tried to use Java for the ETL job at the beginning, but we found that Java is very slow(using multithreads has no improvement of

the speed) and due to the large volume of data, we often encounter run out of memory errors when running the Java code. So we choose scala/spark finally.

- The number and type of instances used during ETL and justification.
  We used 1 n1-highmem-16 master node and 4 n1-highmem-4 worker nodes in our cluster. The reason why we used highmem is that the data volume is so large so we need more memory to store it. We encountered an out of memory error when we didn't use the highmem nodes. In order to make sure that our ETL job can be processed correctly and time-efficiently, we chose to use the highmem nodes.
- The execution time for the ETL process, excluding database load time.
  Executing the ETL process takes about 3 hours.
- The time spent loading your data into the database.
  Loading all tables into the database takes about 6 hours.
- The overall cost of the ETL process.
  About $100 on the GCP team account.
- The number of incomplete ETL runs before your final run.
  We ran lots of incomplete ETL runs(about 50 times). Those were due to the fact that we used lots of rdd.collect() actions in our code.
- The size of the resulting database and reasoning.
  The database is 22G in total, containing three tables. The tweets table is 12G. The score table is 2G. The user table is 8G.
- The size and format of the backup.
  We use a snapshot of the EBS volume hosting the database as a backup. The snapshot is 39G, because this fits into our total budget.
- Discuss the difficulties encountered.
  1. We encountered lots of out of memory errors during the ETL process. Those were caused by either using rdd.collect() lots of time or using low memory nodes.
  2. The spark syntax is nontrivial for spark beginners like us.
  3. It's very time consuming to re-import the data to the database before we were able to use snapshots to recover the database quickly.
  4. The schema needs careful design and implementation.
- If you had multiple database schema iterations, did you have to rerun your whole ETL pipeline between schema iterations?
  We only have 3 iterations. We had to rerun our whole ETL pipeline every time we decided to use the new schema.
- How did you load data into your database? Are there other ways to do this, and what are the advantages of each method? Did you try some other ways to facilitate loading?
  We load data into our database using dataframe.write.jdbc(). This will directly write the data in the dataframe to the database we specified. Another way to do this is using an intermediate csv file to store the dataframe and then load the csv file to the database. But this requires handling of \r, \r\n. We don't think it's a good idea.
- Did you store any intermediate ETL results and why? If so, what data did you store and where did you store the data?

We tried to use csv as intermediate files. But we find it not time-efficient and we need to handle special characters in the csv file. We don't think it's a good idea so we pass on it.

- We would like you to produce a histogram of hashtag frequency on **log** scale among all the valid tweets **without hashtag filtering**. Given this histogram, describe why we asked you to filter out the top hashtags when calculating the hashtag score. [Clarification: We simply need you to plot the hashtag frequencies, sorted on the x-axis by the frequency value on the y-axis]



Based on the histogram, we could observe that these popular hashtags could contribute a large portion of the hashtag score, and make the absolute value really big, which might diminish the contribution of the other two scores, interaction score and keyword score, since their absolute value is way smaller than hashtag score. Thus, we should remove the popular tags to normalize the result.

- Before you run your Spark tasks in a cluster, it would be a good idea to apply some tests. You may choose to test the filter rules, or even test the entire process against a small dataset. You may choose to test against the mini dataset, or design your own dataset that contains interesting edge cases. **Please include a screenshot of the ETL test coverage report here.** (We don't have a strict coverage percentage you need to reach, but we do want to see your efforts in ETL testing.)
  Note: Don't forget to put your code for ETL testing under the **etl** folder in your team's github repo **master** branch.

We designed our own mini dataset(around 10 rows) to test the correctness of our code. Specifically, we added some edge cases such as user1 and user2 that have multiple contacts including replies and retweets. We ran our code on the mini dataset and then checked the result to make sure that it's the same as the result we manually computed. Here are some screenshots of the ETL testing results

```
val interaction_agg_pair_join = spark.sql(
    """
    SELECT _1 AS user_pair, SUM(_2) AS reply_count, SUM(_3) AS retweet_count
    FROM interaction_agg_pair
    GROUP BY user_pair
    ORDER BY reply_count+retweet_count DESC
    """)
interaction_agg_pair_join.show()
```

```
+---------+-----------+-------------+
|user_pair|reply_count|retweet_count|
+---------+-----------+-------------+
|    12 14|          1|            1|
|    11 13|          2|            0|
|    12 13|          0|            1|
|    13 14|          1|            0|
|    11 11|          0|            1|
|    11 14|          1|            0|
|    11 12|          0|            1|
+---------+-----------+-------------+
```

```
interaction_agg_pair_join: org.apache.spark.sql.DataFrame = [user_pair: string, reply_count: bigint ... 1 more field]
```

```
val final_result = spark.sql(
     """
    SELECT interaction._1 AS user1, interaction._2 AS user2, interaction._3 AS interaction_score, hashtag.score AS hashtag_score
    FROM interaction_result AS interaction INNER JOIN hashtag_result AS hashtag
    ON interaction._1 = hashtag.small AND interaction._2 = hashtag.big
    """)
final_result.show()
```

```
+-----+-----+------------------+-------------+
|user1|user2| interaction_score|hashtag_score|
+-----+-----+------------------+-------------+
|   12|   13|0.6931471805599453|          1.0|
|   13|   14|1.0986122886681098|          1.0|
|   11|   14|1.0986122886681098|          1.0|
|   11|   12|0.6931471805599453|          1.0|
|   11|   11|0.6931471805599453|          1.0|
+-----+-----+------------------+-------------+
```

```
final_result: org.apache.spark.sql.DataFrame = [user1: bigint, user2: bigint ... 2 more fields]
```

```
+---------+-------------+---------+---------------------------------+
|user_id_a|all_hashtags_a|user_id_b|all_hashtags_b                   |
+---------+-------------+---------+---------------------------------+
|861      |[CBInsights]  |861      |[CBInsights]                     |
|861      |[CBInsights]  |874      |[WHDemoDay]                      |
|861      |[CBInsights]  |5556     |[AKinAmethi, OurHopeAAP]         |
|861      |[CBInsights]  |5622     |[PackageDesignDrinkingGame]      |
|861      |[CBInsights]  |10399    |[Athnet]                         |
|861      |[CBInsights]  |11569    |[JetsfollowJets]                 |
|861      |[CBInsights]  |11830    |[NHS, coding]                    |
|861      |[CBInsights]  |26253    |[rtfm]                           |
|861      |[CBInsights]  |491123   |[hull]                           |
|861      |[CBInsights]  |625303   |[LFC]                            |
|861      |[CBInsights]  |633443   |[BalloonRelease]                 |
|861      |[CBInsights]  |761906   |[24A, DaleRT]                    |
|861      |[CBInsights]  |768058   |[Jacobin, Merry4Nivôse]          |
|861      |[CBInsights]  |771698   |[e20s]                           |
|861      |[CBInsights]  |800835   |[RIPCarrieFisher]                |
|861      |[CBInsights]  |862771   |[DDoS, defense, deloittepredicts]|
|861      |[CBInsights]  |1123601  |[citizensassembly, repealthe8th] |
|861      |[CBInsights]  |1434881  |[JennyHan, LaraJean]             |
|861      |[CBInsights]  |1448211  |[mPLUSRewards, Nexercise]        |
|861      |[CBInsights]  |1569091  |[SPN, supernatural, lovetheseboys]|
+---------+-------------+---------+---------------------------------+
```

Question 4: Optimizations and Tweaks

Frameworks and databases do not come in the best shape out-of-the-box. Each of them has tens of parameters to tune. Once you are satisfied with the schema, you can start learning about the configuration parameters to see which ones might be most relevant, and gather data and evidence with each individual optimization. You have probably tried a few in Phase 1. To perform even better, you probably want to continue with your experimentation.

Hint: A good schema will easily double or even triple the target RPS of your Microservice 3 without any parameter tuning. It is advised to first focus on improving your schema and get an okay performance with your best cluster configuration. If you are 10 times (an order of magnitude) away from the target, then it is very unlikely to make up the difference with parameter tunings.

- List as many possible items to configure or tweak that might impact performance, in terms of web framework, your program, DBs, DB connectors, OS, etc.
    1. Optimize MySQL query statement, using EXPLAIN statements to inspect the number of rows to be scanned.
    2. Pre-store each user's latest information (screen name, description), instead of searching when requests come.
    3. Deploy 3 MySQL pods instead of one pod.

4. Add cache to all services.

- Apply them one at a time and show a table with the microservice you are optimizing, the optimization you have applied and the RPS before and after applying it.
  We are optimizing m3 service.

Optimization	RPS Before	RPS After
1	286	433
2	433	879
3	879	1028

- For every configuration parameter or tweak, try to explain how it contributes to performance.
  1. If one query scans less number of rows, it executes faster.
  2. Pre-storing each user's latest information avoids doing ORDER BY when querying each user's latest screen name and description, which saves time.
  3. More replicated databases can receive more requests per second.
  4. Caching makes serving the same request faster.

Task 2: Development and Operations

Question 7: Web-tier orchestration development

We know it takes dozens or hundreds of iterations to build a satisfactory system, and the deployment process takes a long time to complete. We asked about the automated deployment process in the Phase 1 Final Report. Answer the following questions to let us know how your automation has changed in Phase 2.

- Did you improve your web-tier deployment process? What were the improvements or changes you made? Include your improved/changed Terraform scripts, Kubernetes manifest or Helm charts under the folders referring to Phase 1 folder structures.

  Helm packages the combined Java web tier or separate ASP.NET Core services. Azure DevOps validates the chart, builds dotnet/vertx/jooby images, pushes Build ID tags to ACR and deploys to AKS. NGINX routes the three endpoints through one LoadBalancer; dev/prod values control replicas.

Question 8: Storage-tier deployment orchestration

In addition to your web-tier deployment orchestration, it is equally important to have automation and orchestration for your storage-tier deployment in order to save labor time and avoid potential human errors during your deployment. Answer the following questions to let us know how your storage-tier deployment orchestration has changed in Phase 2.

- Did you improve your storage-tier orchestration? What were the improvements or changes you made? Include your improved/changed Terraform scripts, Kubernetes manifest or Helm charts under the folders referring to Phase 1 folder structures.

  The Kubernetes storage configuration under k8s/helm uses a StatefulSet with three MySQL pods instead of the single stable/mysql deployment. Data remains separate from the application lifecycle. The Azure delivery path connects m3 to an existing MySQL database using mysqlHost, mysqlUser and mysqlPassword from a variable group and Kubernetes Secret. Its Helm chart manages the web tier; database provisioning, ETL loading and replication remain storage-tier responsibilities.

Question 9: Live test and site reliability

Phase 2 is evaluated differently than how we evaluated Phase 1. In Phase 2, you can make as many attempts as you want before the deadline. However, all these attempts do not contribute to your score. Your team's Phase 2 score is determined by the live test. It is a one-off test of your web service during a specified period of time, and so you will not want anything to suddenly fail; in case you encounter failure, you (or some program/script) probably want to notice it immediately and respond!

- What CloudWatch metrics are useful for monitoring the health of your system? What metrics help you understand performance?
  - Since we used EBS, we monitored some metrics related to EBS, such as VolumeReadOps, VolumeQueueLength(to monitor read/write request latency), BurstBalance(since the EBS volumn is only gp2, we heavily rely on the initial given credit, if the balance become 0%, then the RPS will be low)
  - For load balancer, we monitored unhealthy HTTP response such as HTTPCode_ELB_3/4/5XX_Count.
- Which statistics did you collect when logged into the EC2 VM via SSH? Which tools did you use? What do the statistics tell you?
  - We collected cpu by `top` command and memory usage by `free` command.
- Which statistics did you collect using kubectl? What do the statistics tell you?
  - We collected CPU usage for each pod. It tells us how balance the request load is to each pod.
- How did you monitor the status of your storage-tier? What interesting facts did you observe?
  - We used CloudWatch EBS related metrics such as VolumeReadOps, VolumeQueueLength and BurstBalance.
  - One note is that the process of initialization for newly attached volumn is extremely important. The MySQL query costed 3 second, but only 0.00 second on a initialized volume.
- Should something crash in your system, would (and how) your microservices still be able to serve requests (although perhaps slower)? Think about different scenarios, such as program crashes, network failures or even VM terminations.

  Kubernetes restarts failed containers and removes unready pods from Service endpoints. The Helm chart defines startup, readiness and liveness probes.

A network outage can still interrupt requests. Replicas and node recovery reduce the impact of individual failures. Helm uses --atomic --wait to handle failed rollouts. A failed post-release route check marks the pipeline failed; rollback.sh restores an explicitly selected revision. TCP readiness checks the application listener, so m3 database health needs a query-level check as well.

- During the live test, did anything unexpected happen? If so, how big was the impact on your performance, and what was the reason? What did you do to resolve these issues? Did they affect your overall performance?

  During mixed operations, m3 latency reduced m2 throughput to roughly half its target. This motivates both fewer database round trips and workload isolation. Separate ASP.NET Core containers allow m1, m2 and m3 replicas to be adjusted independently. MySqlConnector uses asynchronous access and per-request pooled connections so m3 requests do not concurrently share one global connection.

Task 4: General questions

Question 10: Scalability

In this phase, we serve read-only requests from tens of GBs of processed data. Remember this is just an infinitesimal slice of all the user-generated content at Twitter. Can your servers provide the same quality of service when the amount of data grows? What if we realistically allow updates to tweets? Here are a few good questions to think about after successfully finishing this phase.

- Would your design work as well if the quantity of data was 10 times larger? What about 100x? Why or why not? (Assume you get a proportional amount of machines.)
  With the appropriate amount of machines, I think our ETL design work could work because the ETL logic remains the same.
- Would your design work if your web service also implemented insert/update (PUT) requests? Why or why not?
  I think we might encounter some problems if insert/update is involved. Because we have some precomputed tables which use historical data. Suppose the data becomes real-time, then these tables are no longer valid and we cannot afford to compute these tables all over again just for one update/insert.
- What kind of changes in your schema design or system architecture would help you serve insert/update requests? Are there any new optimizations that you can think of that you can leverage in this case?
  Since we have 3 MySQL databases in each of the three MySQL pods, consistency is a big issue when it comes to insert/update requests. Maybe we can only allow one primary database to execute insert/update requests, and allow all databases to serve read requests. In this case, the primary database may have a different configuration from secondary databases.

Question 11: Postmortem

- How did you set up your local development environment? Did you install the databases locally on your machines to experiment? Did you test all programs before running them on your Kubernetes cluster on the cloud?
  We installed basic development tools like Intellij, Maven, and cloud CLIs to set up our local development environment. We installed a database service on an GCP instance to experiment. All programs are tested before running them on the k8s cluster.
- Did you attempt to generate a load to test your system on your own? If so, how? And why?
  Yes. We need to profile the Java application and resolve the bottleneck, which will use JMeter to generate loads.
- Describe an alternative design to your system that you wish you had time to try.
  We could have also tried to utilize NoSQL databases such as HBase or MongoDB to store tweets information.
- What were the toughest roadblocks that your team faced in Phase 2?
  The most challenging part in Phase 2 is to optimize ETL process, database schemas, and m3 codes to improve the m3 performance.
- Did you do something unique (any cool optimization/trick/hack) that you would like to share?
  We changed some configurations in *my.cnf* of MySQL database to help accelerate loading data into MySQL database. Otherwise, it might take days to load a large table into the database.

Question 12: Contribution

- In the Phase 1 Final reports, we asked about how you divided the exploration, evaluation, design, development, testing, deployment, etc. components of your system and how you partitioned the work. Were there any changes in responsibilities in Phase 2? Please show us the changes you have made and each member's responsibilities.
  Generally speaking, Qing and Jiarui still work on ETL process (writing scala codes on spark, loading data into databases, joining MySQL tables), and Wangshu still focus on Java backend and deployment (writing yaml files, attaching volumes from snapshots to worker nodes and mounting them to MySQL pods). But for some small parts, we also joined each other and helped solve some problems. For example, Wangshu also helped design the database schema; Qing helped write stateful set yaml files; Jiarui helped integrate the Java codes of three microservices.
- Please show Github stats to reflect the contribution of each team member. Specifically, include screenshots for each of the links below.
  https://github.com/CloudComputingTeamProject/<repo>/graphs/contributors
  https://github.com/CloudComputingTeamProject/<repo>/network



Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to.

